

You are acting as a Principal Software Architect and Staff-plus distributed-systems engineer responsible for extending Correlix, a multi-tenant network observability and causal RCA platform.

Your task is to design and implement first-class, multi-vendor wireless discovery, topology, monitoring, causal correlation, troubleshooting, and guarded auto-healing.

Do not build:

- A disconnected wireless dashboard.
- A collection of vendor-specific scripts.
- A separate wireless incident engine.
- A parallel wireless topology system.
- A new wireless-only evidence or verdict framework.
- An LLM that independently guesses root cause from raw logs.

Wireless must become a native Correlix access domain that connects elegantly into the existing end-to-end service topology:

Wireless Client

→ Wireless Access

→ LAN

→ SD-WAN or WAN

→ Carrier or Cloud Interconnect

→ Data Center or Cloud

→ Application

Correlix must determine whether wireless is:

- The root cause.
- A contributing cause.
- An affected domain.
- Unrelated to the true failure.
- Undetermined because evidence is insufficient.

For example, poor SaaS performance experienced by wireless users must not be classified as a wireless issue when the true cause is:

- AWS Direct Connect packet loss.
- Azure ExpressRoute impairment.
- ISP circuit degradation.
- SD-WAN tunnel instability.
- LAN congestion.
- DNS failure.
- Application degradation.
- Cloud-provider service impairment.

The implementation must preserve Correlix's core thesis:

Identify the actual causal failure domain across infrastructure layers without confidently blaming the closest alert.

1. Mandatory operating rules

Follow these rules throughout the work:

1. Inspect the repository thoroughly before making production changes.
2. Reuse existing Correlix abstractions wherever appropriate.
3. Do not create parallel discovery, topology, correlation, evidence, incident, workflow, ticketing, or remediation frameworks.
4. Do not assume the existing implementation is correct merely because tests pass.
5. Test real behavior, not only source-code strings or mocked interfaces.
6. Preserve strict multi-tenant isolation.
7. Preserve source event-time causal ordering.
8. Do not silently substitute receive time for event time.
9. Do not confirm causality from temporal proximity alone.
10. Do not let an LLM directly construct and execute arbitrary device commands.
11. Prefer an `UNDETERMINED` verdict over an unsupported confident RCA.
12. Make schemas versioned and backward-compatible where reasonably possible.
13. Deliver changes in small, independently reviewable phases.
14. Do not begin broad implementation until the architecture and repository-impact report are complete.
15. Clearly separate verified facts, architectural inference, assumptions, and areas requiring live validation.
16. Do not claim production readiness merely because code compiles or mocked tests pass.
17. Build tests for failure classes, not only individual examples.
18. Preserve original vendor timestamps and raw evidence.
19. Treat unsupported provider capabilities as explicit states, not generic failures.
20. Do not allow vendor-specific enums to leak directly into canonical schemas.

2. Mandatory first-pass repository investigation

Before implementing production code, inspect and document the current Correlix architecture.

Locate the exact files, packages, functions, types, interfaces, database tables, event topics, APIs, background workers, and frontend components responsible for:

- Device discovery.
- Provider or adapter registration.
- SNMP polling.
- SNMP traps.
- gNMI collection.
- NETCONF collection.
- Syslog ingestion.

- Flow ingestion.
- Vendor API integrations.
- Active probing.
- LAN topology.
- Access-switch modeling.
- Switch-port modeling.
- Endpoint identity.
- Seam inventory.
- Path definitions.
- Path observations.
- Ordered path hops.
- Topology objects.
- Topology edges.
- Correlation signals.
- Correlation objects.
- Correlation-object identity.
- Correlation-object versioning.
- Correlation grouping.
- Correlation windows.
- Correlation edges.
- Evidence storage.
- Supporting evidence.
- Contradicting evidence.
- Missing evidence.
- Verdict calculation.
- Verdict publication.
- Incident merge.
- Canonical-survivor handling.
- Incident resolution.
- Late-event handling.
- Incident reopen protection.
- Timestamp normalization.
- Event-time and receive-time handling.
- Clock-skew handling.
- Redpanda or Kafka topics.
- Message schemas.
- ClickHouse writes and reads.
- PostgreSQL state.
- Redis keys and cache behavior.
- VictoriaMetrics metrics.
- OpenSearch logs.
- Tenant authorization.
- ClickHouse row policies.
- PostgreSQL tenant predicates.
- Reporting.
- Notifications.
- Ticketing.
- Transactional outbox processing.

- Existing remediation or action workflows.
- Iris AI and RCA entry points.
- API routes.
- Frontend topology views.
- Frontend correlation views.
- Test fixtures.
- Contract tests.
- Integration tests.
- Fault-injection infrastructure.
- Feature flags.
- Migration conventions.
- Deployment and rollback conventions.

Search for every enum, switch statement, validator, database constraint, API schema, serializer, event consumer, and UI mapping that assumes a fixed list of:

- Seam types.
- Object types.
- Edge types.
- Signal types.
- Evidence types.
- Verdict types.
- Provider types.
- Health dimensions.
- Path-hop types.
- Remediation actions.

Produce an initial report containing:

1. Existing abstractions that should be reused.
2. Existing abstractions that require extension.
3. Existing abstractions that are unsafe or incomplete.
4. Places where wireless would currently be incorrectly forced into a wired model.
5. Architectural blockers.
6. Multi-tenancy risks.
7. Event-time and causal-ordering risks.
8. High-cardinality storage risks.
9. Schema and migration impact.
10. Exact expected files and packages to change.
11. Recommended implementation sequence.
12. Decisions requiring approval.
13. Areas requiring live-controller validation.
14. Areas requiring physical fault injection.

Do not make broad production changes during this initial investigation.

Documentation changes, non-invasive inspection utilities, and tests that expose current behavior are allowed.

Create or update:

- docs/design/wireless-domain-architecture.md
- docs/design/wireless-discovery.md
- docs/design/wireless-correlation.md
- docs/design/wireless-iris-remediation.md
- docs/design/wireless-testing-rollout.md

3. Mandatory Correlix correlation-engine integration

Wireless must extend the existing Correlix correlation lifecycle.

Do not create separate tables or systems such as:

- wireless_incidents
- wireless_correlations
- wireless_evidence
- wireless_verdicts
- wireless_ticket_links
- wireless_notifications

unless repository analysis proves an unavoidable need and the decision is explicitly approved.

The intended flow is:

Wireless observations

- Existing normalized signal-ingestion framework
- Existing `corr_signals` or repository equivalent
- Existing correlation grouping and temporal windows
- Existing `corr_objects` or repository equivalent
- Existing correlation edges and evidence
- Existing verdict lifecycle
- Existing reports, notifications, ticketing, and Inspector UI

Document exactly how wireless maps into:

- Existing signal ingestion.
- Signal deduplication.
- Correlation windows.
- Correlation-object identity.
- Correlation-object versioning.
- Incident merge behavior.
- Canonical-survivor behavior.
- Correlation edges.

- Supporting evidence.
- Contradicting evidence.
- Missing evidence.
- Verdict calculation.
- Resolution.
- Late-event handling.
- Reopen protection.
- Reporting.
- Ticketing.
- Notifications.
- Transactional outbox behavior.

Preserve the existing Correlix principle that a root cause is not **CONFIRMED** merely because one source produced several related alerts.

Two syslog messages from one controller are not two independent modalities.

Examples of genuinely independent modalities include:

- Switch PoE telemetry plus AP control-session loss.
- Switch-port link state plus AP reachability loss.
- RADIUS transaction evidence plus client onboarding-stage evidence.
- Data-tunnel telemetry plus synthetic service probe failure.
- RF measurements plus retry or frame-delivery evidence.
- Controller state plus path telemetry.
- DHCP transaction evidence plus client address-state evidence.
- WAN telemetry plus wired and wireless probe failures.

Use verdict semantics equivalent to:

CONFIRMED

At least two genuinely independent evidence modalities support the same causal chain, and no decisive contradiction exists.

SUSPECTED

A plausible causal chain exists, but corroboration is incomplete.

UNDETERMINED

Evidence cannot reliably distinguish competing causes.

Do not publish an Iris narrative with higher certainty than the structured verdict.

4. Relationship with the existing LAN seam

Wireless should be introduced as a specialized access domain within or directly attached to the existing LAN seam.

Preferred conceptual model:

LAN

|—— Wired Access
|—— Wireless Access

Wireless should reuse the existing LAN seam after the wireless forwarding boundary.

Examples:

- A locally switched client joins the LAN topology at the AP uplink and VLAN.
- A centrally switched client joins the LAN topology at the controller or gateway tunnel-termination VLAN.
- The AP management path joins LAN through its management VLAN, gateway, and switch-port relationship.
- A controller in a data center must connect to the existing DC and WAN topology.
- A cloud-managed AP must connect to the vendor cloud through the existing WAN, firewall, DNS, and Internet dependency topology.
- A controller or gateway must not be modeled as an isolated wireless appliance.

Do not create a new top-level seam unless repository analysis proves the current seam architecture cannot cleanly support a wireless access subdomain.

Document the decision explicitly.

5. Core architectural principle: multiple simultaneous graphs

Wireless cannot be modeled as merely another access interface.

Correlix must maintain and correlate several separate but connected graphs.

5.1 Wireless client service-path graph

Locally bridged or locally switched traffic

Wireless client session

→ Wireless client link

→ BSSID

- Radio
- Access point
- AP Ethernet interface
- Access-switch port
- Access switch
- VLAN or VRF
- Default gateway
- Existing LAN, SD-WAN, carrier, cloud, DC, and application path

Centrally tunneled traffic

Wireless client session

- Wireless client link
- BSSID
- Radio
- Access point
- Wireless data tunnel
- Wireless gateway or controller
- Tunnel-termination VLAN or VRF
- Existing LAN, SD-WAN, carrier, cloud, DC, and application path

Mixed forwarding

A single WLAN or deployment may have:

- Some locally forwarded traffic.
- Some centrally tunneled traffic.
- Split-tunnel policies.
- Role-dependent forwarding.
- Application-dependent forwarding.
- Fabric encapsulation after wireless tunnel termination.

The model must support these without creating fake topology.

5.2 Wireless infrastructure dependency graph

Access point

- Power source
- AP interface
- Switch port
- Access switch
- Management VLAN or VRF
- Default gateway
- Firewall
- DNS
- NTP

- Controller discovery
- Controller, gateway, virtual controller, or vendor cloud

5.3 Client onboarding dependency graph

Wireless client session

- WLAN
- Security policy
- AAA profile
- RADIUS or identity service
- Certificate or PKI service
- DHCP service
- DNS service
- Captive portal, when applicable
- Policy engine
- Mobility anchor, when applicable

5.4 Wireless RF relationship graph

Radio

- RF neighbor
- Channel
- Band
- RF domain
- Interference source
- Coverage relationship

This graph should not be confused with the ordered service path.

5.5 Ordered service path versus general topology

Do not derive an ordered service path merely because objects are adjacent in the topology graph.

Wireless must extend the existing ordered path model, including repository equivalents of:

- PathDefinition
- PathObservation
- PathHop
- PATH_HAS_HOP
- CROSSES_SEAM
- HOP_RESOLVES_TO
- TERMINATES_AT_ENDPOINT
- EVIDENCE_SUPPORTS
- EVIDENCE_CONTRADICTS
- EVIDENCE_MISSING

The wireless portion should become an ordered path prefix attached to the existing LAN and end-to-end service path.

6. Vendor-neutral canonical object model

Design and implement vendor-neutral canonical object types equivalent to:

- wireless_ap
- wireless_ap_interface
- wireless_radio
- wireless_bssid
- wireless_ssid
- wireless_wlan
- wireless_client
- wireless_client_session
- wireless_client_link
- wireless_client_mld
- wireless_ap_mld
- wireless_controller
- wireless_controller_cluster
- wireless_controller_member
- wireless_gateway
- wireless_gateway_cluster
- wireless_gateway_member
- wireless_control_domain
- wireless_manager
- wireless_control_session
- wireless_data_tunnel
- wireless_tunnel_endpoint
- wireless_mobility_domain
- wireless_rf_domain
- wireless_policy
- wireless_security_profile
- aaa_profile
- aaa_service
- dhcp_service
- dns_service
- captive_portal_service

Adapt names to existing repository conventions.

Do not place vendor names such as Cisco, Aruba, Mist, Meraki, Ruckus, or Extreme into canonical object-type names.

Vendor-specific terminology must be represented through:

- Vendor.
- Platform.
- Product family.
- Software version.
- Capability metadata.
- Protocol.
- Product role.
- Source attributes.
- Versioned vendor extensions.

7. Separate SSID, WLAN, and BSSID

Do not treat SSID, WLAN, and BSSID as interchangeable.

Model:

`wireless_ssid`

The advertised network name.

`wireless_wlan`

The configured wireless service binding:

- SSID.
- Security profile.
- AAA profile.
- Policy.
- VLAN or segment.
- Forwarding mode.
- QoS.
- Site assignment.
- Radio availability.
- Tunnel behavior.

`wireless_bssid`

A radio and AP-specific instantiation of a wireless service.

Relationships should support:

Wireless WLAN `advertises` SSID

Wireless BSSID `instantiates` WLAN

Wireless WLAN `uses_policy` wireless policy
Wireless WLAN `uses_security_profile` security profile
Wireless WLAN `uses_aaa_profile` AAA profile
Wireless WLAN `maps_to` VLAN, VRF, segment, or tunnel policy

Do not use SSID text as the stable identity of a WLAN.

8. Wi-Fi 7 Multi-Link Operation support

Do not assume one wireless session has exactly one radio, BSSID, band, or physical link.

The canonical model must support Wi-Fi 7 Multi-Link Operation without requiring a future breaking migration.

Model:

- Wireless client MLD.
- AP MLD.
- Logical wireless client session.
- One-to-many wireless client links.
- Link-level BSSID.
- Link-level radio.
- Link-level band.
- Link-level channel.
- Link-level state.
- Link-level metrics.
- Aggregated MLD-level state.
- Aggregated MLD-level metrics.
- Link-level roaming.
- Session-level roaming.

Conceptual relationship:

Wireless client

→ has session

Wireless client session

→ has one or more wireless client links

Wireless client link

→ associated with BSSID

→ uses radio

→ terminates on AP MLD or AP

A session should support fields equivalent to:

```
session_id:
tenant_id:
client_id:
client_mld_id:
ap_mld_id:
mlo_enabled:
mlo_mode:
identity:
username:
device_profile:
wlan_id:
ssid_id:
security_profile_id:
aaa_profile_id:
vlan_id:
vrf_id:
segment_id:
ip_addresses:
association_state:
authentication_state:
authorization_state:
addressing_state:
dns_state:
forwarding_mode:
controller_id:
gateway_id:
data_tunnel_id:
mobility_anchor_id:
source_event_time:
valid_from:
valid_to:
disconnect_reason:
links:
  - link_id:
    client_link_mac:
    ap_link_mac:
    bssid_id:
    radio_id:
    band:
    channel:
    state:
    valid_from:
    valid_to:
    per_link_metrics:
```

Roaming may occur:

- At the logical session level.
- At the individual link level.
- On one band while another link remains active.

Do not flatten MLO links into independent clients.

9. Wireless client identity and temporal sessions

Do not model a wireless client solely as a persistent MAC-address endpoint.

A client can:

- Roam between APs.
- Move between BSSIDs.
- Reauthenticate.
- Change VLAN or role.
- Change IP address.
- Use randomized MAC addresses.
- Disconnect and reconnect.
- Move between local and centralized forwarding.
- Fail before receiving an IP address.
- Maintain identity while changing radio attachment.
- Use multiple MLO links simultaneously.

Create a temporal `wireless_client_session`.

Where available, preserve:

- Tenant ID.
- Session ID.
- Endpoint ID.
- Client MLD identity.
- Client MAC or link MACs.
- Stable identity.
- Username.
- Device profile.
- AP or AP MLD.
- Radio.
- BSSID.
- SSID.
- WLAN.
- VLAN.
- VRF.
- Segment.
- IP addresses.

- Association state.
- Authentication state.
- Authorization state.
- Addressing state.
- DNS state.
- Forwarding mode.
- Controller.
- Gateway.
- Data tunnel.
- Mobility anchor.
- Source event-time start.
- Source event-time end.
- Normalized event time.
- Disconnect reason.
- Previous AP.
- Target AP.
- Vendor session ID.

Define explicit identity precedence and reconciliation rules for:

- Vendor cloud resource ID.
- Controller resource ID.
- Serial number.
- Base MAC.
- MLD MAC.
- Radio MAC.
- Management IP.
- Hostname.
- LLDP identity.
- Switch-port relationship.
- Client username.
- Certificate identity.
- DHCP identity.
- Endpoint fingerprint.

All identities must be deterministic, stable, and tenant-scoped.

10. Canonical topology edges

Support normalized edges equivalent to:

- Client `has_session` client session.
- Client session `has_link` client link.
- Client link `associated_with` BSSID.
- BSSID `advertised_by` radio.
- BSSID `instantiates` WLAN.
- WLAN `advertises` SSID.

- Radio `contained_by` AP.
- Radio `member_of` AP MLD.
- AP `has_interface` AP interface.
- AP interface `connected_to` switch port.
- AP interface `member_of` LAG.
- AP `powered_by` switch port or PoE source.
- AP `managed_by` wireless manager.
- AP `controlled_by` control domain.
- AP `has_control_session` control session.
- AP `has_data_tunnel` data tunnel.
- Data tunnel `terminates_on` tunnel endpoint.
- Tunnel endpoint `hosted_by` controller or gateway member.
- Controller member `member_of` controller cluster.
- Gateway member `member_of` gateway cluster.
- Client session `authenticated_by` AAA service.
- Client session `authorized_by` wireless policy.
- Client session `addressed_by` DHCP service.
- Client session `resolved_by` DNS service.
- Client session `uses_captive_portal` captive portal service.
- Client session `forwarded_through` data tunnel.
- Client session `locally_bridged_to` VLAN or segment.
- Client session `anchored_on` gateway or controller member.
- AP `member_of` mobility domain.
- AP `rf_neighbor_of` AP.
- Radio `rf_neighbor_of` radio.

Every dynamic edge must support:

- Tenant ID.
- Source event time.
- Normalized event time.
- Valid-from time.
- Valid-to time.
- Observation source.
- Evidence references.
- Confidence.
- Source sequence.
- Normalization version.
- Vendor object reference.
- Field-level provenance.
- Timestamp uncertainty.

Roaming must close or supersede the previous temporal association edge and create a new edge.

Do not overwrite roaming history in place.

11. Logical control domains, clusters, and physical members

Represent separately:

- Logical wireless control domain.
- Controller cluster.
- Controller member.
- Gateway cluster.
- Gateway member.
- Configured primary cluster.
- Configured secondary cluster.
- Actual control-session peer.
- Actual tunnel endpoint.
- Active member.
- Standby member.
- Client anchor member.
- Mobility anchor.
- Cluster health.
- Member health.

Do not infer a cluster failure from the failure of one member.

Do not treat a planned failover, member failure, client re-anchoring, and complete cluster failure as the same event.

An AP may maintain control with one member while client data tunnels terminate on another system.

12. Wireless protocols, encapsulation, and forwarding modes

Do not assume AP-to-controller communication universally uses L2TP.

The model must represent:

- CAPWAP control.
- CAPWAP data.
- DTLS-protected CAPWAP.
- GRE.
- IPsec.
- L2TPv3 where actually applicable.
- Local bridging.
- Central tunneling.

- Split tunneling.
- Mixed forwarding.
- Decrypt-tunnel forwarding.
- Cloud-management sessions.
- Controllerless operation.
- Virtual-controller behavior.
- Mesh backhaul.
- Mobility-control sessions.
- VXLAN.
- Campus-fabric encapsulation.
- User-based tunneling.
- Nested encapsulation.

Separate:

- Management session.
- Control session.
- Client data tunnel.
- Mobility session.
- Cloud-management session.

Represent an encapsulation stack where necessary:

```
forwarding_context:
  mode:
  vlan:
  vrf:
  segment_id:
  policy_group:
  encapsulation_stack:
    - capwap
    - gre
    - ipsec
    - vxlan
```

A control session may be healthy while the client data tunnel is impaired.

A cloud-management session may be down while local client forwarding remains functional.

A controller may be reachable while AP authentication, certificate validation, or data-tunnel establishment fails.

13. AP uplink and power model

Do not place one `switch_port_id` directly on the AP and assume it is authoritative.

An AP may have:

- Multiple Ethernet interfaces.
- Link aggregation.
- Backup Ethernet.
- Mesh backhaul.
- Cellular or alternate uplink.
- PoE on one port while data uses an aggregate.
- Reduced radio capabilities because of insufficient PoE.
- One management interface and several forwarding interfaces.

Support relationships equivalent to:

Wireless AP

→ has AP interface

AP interface

→ member of LAG

→ connected to switch port

→ powered by PoE source

→ used as primary, secondary, mesh, or backup uplink

Model power state separately from data-link state.

Support conditions such as:

- Link up but insufficient PoE.
- AP operating in reduced-power mode.
- One radio disabled because of power budget.
- LAG member failure without full AP outage.
- Mesh fallback after Ethernet failure.

14. Capability-driven provider architecture

Fit wireless integrations into existing Correlix provider and discovery abstractions where possible.

Avoid one oversized provider interface that every vendor must implement.

Prefer capability groups equivalent to:

- `WirelessInventoryProvider`
- `WirelessStateProvider`
- `WirelessEventProvider`
- `WirelessTelemetryProvider`
- `WirelessDiagnosticProvider`
- `WirelessActionProvider`

Alternatively, preserve one provider root with independently registered sub-capabilities.

Capabilities should include:

- Controller inventory.
- Gateway inventory.
- Manager inventory.
- Cluster inventory.
- AP inventory.
- AP-interface inventory.
- AP MLD inventory.
- Radio inventory.
- SSID inventory.
- WLAN inventory.
- BSSID inventory.
- Client-session inventory.
- Client-link inventory.
- MLO support.
- Client lifecycle events.
- AP join state.
- Control-session state.
- Data-tunnel state.
- Mobility state.
- RF metrics.
- Packet-capture capability.
- Client connectivity tests.
- Client deauthentication.
- Radio restart.
- AP restart.
- Channel modification.
- Transmit-power modification.
- Webhooks.
- Streaming telemetry.
- NETCONF.
- RESTCONF.
- gNMI.
- SNMP.
- Syslog.
- Cloud API.

Capabilities must be explicit.

An unsupported capability is a valid provider result, not a generic provider error.

Do not leak vendor-specific structures into correlation logic.

All vendor inputs must pass through a canonical normalization layer.

Source preference must be defined per capability and per field, not through one universal source ranking.

15. Field-level provenance and conflict resolution

Every normalized field and relationship must preserve provenance.

Use a structure equivalent to:

```
value:
source:
source_object_id:
source_method:
observed_at:
valid_from:
valid_to:
confidence:
authority:
normalization_version:
```

The same AP field may be learned from:

- Controller YANG.
- Controller REST API.
- Vendor cloud API.
- AP-local API.
- SNMP.
- AP-reported LLDP.
- Switch-reported LLDP.
- Switch MAC table.
- Syslog.
- Manual inventory.

Define conflict-resolution rules per field.

Example: AP uplink switch port should be considered strongly confirmed when:

- AP-reported LLDP identifies the switch and port.
- Switch-reported LLDP identifies the AP.
- AP MAC or interface MAC is learned on that port.

One-sided LLDP should remain lower confidence.

Do not overwrite conflicting values without retaining provenance.

16. Initial vendor implementation: Cisco Catalyst 9800

Implement Cisco Catalyst 9800 as the first complete provider while ensuring the architecture is not Cisco-specific.

Prefer structured sources based on the capability being collected.

Potential sources include:

- NETCONF/YANG.
- RESTCONF.
- gNMI.
- Model-driven telemetry.
- Structured REST APIs.
- SNMP.
- Traps.
- Syslog.
- CLI only as a controlled fallback.

Support, where the platform exposes it:

- Controller inventory.
- Controller HA state.
- Controller cluster membership.
- AP inventory.
- AP interfaces.
- AP MLD state.
- AP join state.
- AP join failures.
- CAPWAP control state.
- CAPWAP data state.
- Local switching.
- Central switching.
- Mixed forwarding.
- Radios.
- SSIDs.
- WLANs.
- BSSIDs.
- Wireless clients.
- Client links.
- MLO state.
- Association state.
- Authentication and EAPOL state.
- Authorization state.

- DHCP and IP state.
- Roaming and mobility.
- RRM.
- RF measurements.
- Disconnect reasons.
- Controller data-plane health.
- Telemetry health.

Create sanitized golden fixtures from realistic Catalyst 9800 payloads.

Clearly label every capability as:

- Implemented and unit tested.
- Tested with recorded payloads.
- Tested against a simulator.
- Tested against a live controller.
- Tested with physical fault injection.
- Not yet validated.

Do not claim complete Catalyst 9800 support based only on schema or source-code inspection.

17. Multi-vendor architecture proof

After defining the Cisco provider and canonical contracts, prove that the architecture can represent substantially different wireless systems.

Create an Aruba Central or ArubaOS 10 provider skeleton or limited implementation demonstrating support for:

- Cloud management.
- AP-to-gateway control.
- Gateway clusters.
- GRE client-data tunnels.
- IPsec control relationships where applicable.
- Local forwarding.
- Tunnel forwarding.
- Mixed forwarding.
- Client anchoring.
- Gateway-member failover.

Validate that the canonical model can also represent cloud-managed, locally forwarding architectures such as Mist or Meraki without inventing a fake on-premises controller.

The architecture must represent:

- Cisco controller-based CAPWAP.
- Aruba gateway-tunneled wireless.

- Cloud-managed local forwarding.
- Controllerless systems.
- Local switching.
- Central switching.
- Mixed forwarding.
- AP management paths separate from client data paths.
- Logical clusters separate from physical members.
- Wi-Fi 7 MLO.

Do not merge the canonical model if it works elegantly only for Cisco.

18. Canonical observation envelope

Create a versioned wireless observation schema integrated with the existing Correlix event framework.

It must preserve:

- Schema version.
- Normalization version.
- Tenant ID.
- Observation ID.
- Observation type.
- Source event time.
- Collector-observed time.
- Broker-ingest time.
- Correlix-ingest time.
- Normalized event time.
- Receive time where already required.
- Source vendor.
- Source platform.
- Source controller, gateway, cloud, or device.
- Collector type.
- Source sequence.
- Estimated clock offset.
- Timestamp uncertainty.
- Source clock status.
- Normalization method.
- Subject object.
- Related objects.
- Normalized attributes.
- Field-level provenance.
- Raw evidence reference.

Never overwrite the original vendor timestamp.

Create event families equivalent to:

- ap.discovery.*
- ap.join.*
- ap.control_session.*
- ap.data_tunnel.*
- ap.uplink.*
- ap.power.*
- ap.reboot.*
- ap.image.*
- ap.config.*
- ap.radio.*
- ap.mlo.*
- client.probe.*
- client.association.*
- client.authentication.*
- client.authorization.*
- client.addressing.*
- client.dhcp.*
- client.ipv6.*
- client.dns.*
- client.captive_portal.*
- client.roam.*
- client.mlo.*
- client.disconnect.*
- client.performance.*
- rf.channel.*
- rf.interference.*
- rf.noise.*
- rf.utilization.*
- rf.coverage.*
- rf.rogue.*
- controller.health.*
- controller.ha.*
- controller.mobility.*
- controller.capacity.*
- gateway.health.*
- gateway.cluster.*
- wireless.policy.*
- wireless.cloud_connectivity.*

Use normalized source event time for causal ordering.

Receive time may be used for transport-delay analysis but must not silently determine causality.

Support causal-order relationships equivalent to:

- Definitely before.
- Probably before.
- Concurrent within uncertainty.
- Order indeterminate.
- Contradicted by source sequence.

Explicitly handle:

- Clock skew.
- Missing source timestamps.
- Timestamp precision.
- Timestamp uncertainty.
- Source sequence.
- Out-of-order events.
- Duplicate events.
- Delayed delivery.
- Replayed events.
- Controller failover.
- Collector restart.
- Cloud API delay.
- Batch polling timestamps.

Add tests proving the actual timestamp-normalization code path is exercised end to end.

Do not accept source-scan-only validation.

19. Applicability-aware client onboarding episode

Do not implement onboarding as one mandatory linear sequence.

Create an applicability-aware onboarding episode.

Potential stages include:

- Seen.
- Probing.
- Association.
- Layer-2 associated.
- Authentication.
- EAP processing.
- Certificate validation.
- Authorization.
- Policy application.
- IPv4 addressing.

- IPv6 addressing.
- DHCPv4.
- DHCPv6.
- SLAAC.
- DNS readiness.
- Captive portal.
- Service validation.

Support:

- WPA2 Personal.
- WPA3 Personal.
- SAE.
- OWE.
- 802.1X.
- EAP-TLS.
- PEAP.
- MAC authentication.
- Passpoint.
- OpenRoaming.
- Static IPv4.
- DHCPv4.
- IPv6 SLAAC.
- DHCPv6.
- Dual stack.
- Captive portal.
- Cached authentication.
- Fast roaming.
- Reauthentication.
- MLO link addition and removal.

An onboarding episode should contain:

```
episode_id:
session_id:
authentication_method:
addressing_mode:
applicable_stages:
completed_stages:
failed_stage:
skipped_stages:
retry_count:
stage_attempts:
last_successful_stage:
first_failed_stage:
```

```
supporting_evidence:  
contradicting_evidence:
```

A stage that is not applicable must not be classified as missing or failed.

Support failure states equivalent to:

- Association rejected.
- Authentication timeout.
- Authentication rejected.
- Certificate failure.
- Policy application failure.
- DHCP discover with no offer.
- DHCP request with no ACK.
- DHCPv6 failure.
- SLAAC failure.
- Duplicate IP.
- DNS unreachable.
- Captive portal failure.
- Data-path unreachable.
- MLO link establishment failure.
- Service validation failure.

Each stage transition must include supporting evidence.

Determine:

- Last successful stage.
- First failed stage.
- Time spent in each stage.
- Retry count.
- Shared failure domain.
- Affected-client scope.
- Contradicting evidence.

Do not declare RADIUS, DHCP, DNS, the AP, or the controller as root cause merely because a nearby alert exists.

Require stage-specific and path-specific evidence.

20. Correlation signatures

Implement deterministic wireless correlation signatures before Iris performs language-model reasoning.

Each signature must define:

- Required evidence.
- Optional evidence.
- Contradicting evidence.
- Missing evidence.
- Event-time window.
- Timestamp-uncertainty rules.
- Topology boundary.
- Shared-dependency rules.
- Minimum independent modalities.
- Confidence policy.
- Affected-scope calculation.
- Resolution criteria.
- Reopen criteria.
- Late-event behavior.

20.1 AP offline differentiation

Distinguish:

- PoE loss.
- Insufficient PoE.
- Reduced-power mode.
- Switch-port down.
- LAG member failure.
- Access-switch failure.
- AP reboot.
- AP software crash.
- Management VLAN failure.
- Management gateway failure.
- DNS failure.
- NTP or certificate-time failure.
- Controller-discovery failure.
- CAPWAP or control-protocol failure.
- Certificate or DTLS failure.
- Controller capacity issue.
- Controller-member failure.
- Controller-cluster failure.
- WAN-path failure.
- Firewall block.
- Vendor cloud-management failure.
- Ethernet-to-mesh failover.

Example causal chain:

PoE removed

→ AP loses power

→ control session disappears

→ radios disappear

→ wireless clients disconnect

The root cause should be the PoE or switch-port failure, not the controller-session alarm.

20.2 Mass AP control-session loss

Group AP failures by shared:

- Site.
- Switch.
- Power domain.
- Management VLAN.
- VRF.
- Gateway.
- Firewall.
- SD-WAN path.
- ISP circuit.
- Controller member.
- Controller cluster.
- Gateway cluster.
- Cloud endpoint.
- DNS dependency.

Example:

Multiple APs lose controller sessions

- AP Ethernet links remain up
- controller cluster remains healthy
- management gateway becomes unreachable
- SD-WAN loss begins at the same event time

Likely cause:

Site management path or WAN failure.

20.3 Client onboarding failure

Identify:

- Applicable stages.
- Last successful stage.

- First failed stage.
- Retry sequence.
- Shared dependency.
- Contradicting successful clients.

Differentiate:

- RF discovery failure.
- Association rejection.
- SAE failure.
- 802.1X failure.
- RADIUS timeout.
- RADIUS rejection.
- Certificate failure.
- Policy-assignment failure.
- DHCPv4 failure.
- DHCPv6 failure.
- SLAAC failure.
- DNS failure.
- Captive-portal failure.
- Post-onboarding data-path failure.

20.4 Wireless performance degradation

Differentiate:

- Weak signal.
- Low SNR.
- RF interference.
- Channel congestion.
- Retry storm.
- Low PHY rate.
- MLO link imbalance.
- MLO link failure.
- AP radio exhaustion.
- AP CPU or memory.
- AP Ethernet-uplink errors.
- AP uplink congestion.
- Central data-tunnel impairment.
- Gateway data-plane impairment.
- Controller data-plane impairment.
- LAN congestion.
- SD-WAN impairment.
- WAN impairment.
- Carrier impairment.
- AWS Direct Connect impairment.
- Azure ExpressRoute impairment.

- Internet-circuit impairment.
- Cloud impairment.
- Application impairment.

Wireless must be allowed to be an affected domain rather than the root cause.

20.5 Roaming failure

Differentiate:

- Sticky-client behavior.
- Insufficient target coverage.
- Target AP unavailable.
- Target AP capacity.
- Authentication delay.
- Fast-transition failure.
- Key-caching issue.
- Mobility-anchor failure.
- Controller or gateway control-plane latency.
- WAN latency.
- Upstream service interruption.
- MLO link-transition failure.

20.6 AP control healthy, data path unhealthy

Detect situations where:

- AP control session remains up.
- AP radios remain operational.
- Client association succeeds.
- Client data tunnel is impaired.
- Client service probes fail.

Do not declare AP or controller-control failure.

20.7 Cloud management down, forwarding healthy

Detect situations where:

- Vendor cloud management is unreachable.
- Local client forwarding remains healthy.
- AP radios remain operational.
- Client service probes succeed.

Classify cloud management as degraded without overstating customer impact.

21. Wireless health model

Do not create one opaque wireless score.

Produce separately explainable dimensions:

- RF health.
- Onboarding health.
- AP uplink health.
- AP power health.
- Control-plane health.
- Data-tunnel health.
- Controller health.
- Gateway health.
- Cloud-management health.
- Service-path health.
- Roaming health.
- MLO health.
- Client-experience health.

An overall score may be derived, but it must include:

- Limiting dimension.
- Evidence that reduced the score.
- Affected scope.
- Confidence.
- Whether the issue originates inside or outside wireless.
- Whether the issue affects control, management, data, onboarding, or RF only.

22. High-cardinality storage strategy

Do not materialize every client sample, RF measurement, probe, retry, or association attempt as a permanent topology object or edge.

Define explicit storage placement and retention.

Conceptually:

PostgreSQL

Use for durable configuration and control-plane state, such as:

- Providers.
- Integrations.
- Controllers.

- Gateways.
- AP inventory.
- WLAN definitions.
- Policies.
- Credentials references.
- Remediation policies.
- Approval policies.

ClickHouse

Use for high-volume temporal and correlation data, such as:

- Client sessions.
- Client links.
- Onboarding episodes.
- Wireless events.
- Correlation signals.
- Evidence.
- Temporal relationships.
- Historical state.
- Action history where appropriate.

VictoriaMetrics

Use for time-series data, such as:

- RF metrics.
- Radio metrics.
- AP health metrics.
- Controller metrics.
- Tunnel metrics.
- Client performance metrics.
- MLO link metrics.

OpenSearch

Use for relevant logs and searchable raw event context.

Materialized topology

Use primarily for:

- Durable infrastructure.
- Active or incident-relevant client relationships.
- On-demand client paths.
- Aggregated relationships.

Do not load every historical wireless client into the default topology canvas.

Support on-demand materialization by:

- Client.
- Session.
- AP.
- WLAN.
- Site.
- Incident.
- Failure signature.

Provide scale estimates for:

- Tenants.
- Sites.
- Controllers.
- APs.
- Radios.
- BSSIDs.
- WLANs.
- Concurrent clients.
- Client links.
- Events per second.
- Telemetry samples per second.
- Retention.
- Storage growth.
- Query latency.

23. Iris troubleshooting integration

Iris must consume structured Correlix state.

It must not infer root cause directly from raw logs.

Use this sequence:

Deterministic state reconstruction

- Candidate failure-domain generation
- Evidence-graph evaluation
- Contradiction checks
- Counterfactual checks
- Confidence-calibrated diagnosis
- Troubleshooting plan
- Guarded remediation eligibility

Iris output must distinguish:

- Observed fact.
- Derived state.
- Correlated relationship.
- Hypothesis.
- Suspected cause.
- Confirmed cause.
- Recommended action.
- Approved action.
- Executed action.
- Validated recovery.

Return structured results containing:

- Incident ID.
- Verdict.
- Confidence.
- Root-cause object.
- Root-cause class.
- Responsibility seam.
- Affected scope.
- Causal chain.
- Supporting evidence.
- Contradicting evidence.
- Missing evidence.
- Unknowns.
- Recommended diagnostic checks.
- Recommended remediation.
- Remediation risk.
- Approval requirement.
- Validation criteria.

When evidence is inadequate, return `UNDETERMINED`.

Do not generate a confident narrative RCA when the structured verdict is undetermined.

24. Iris troubleshooting workflows

Design or implement structured troubleshooting workflows for:

- AP not joining controller.
- AP repeatedly joining and disconnecting.
- AP offline.
- AP control session down.
- AP data tunnel down.
- AP management-connectivity loss.

- AP radio down.
- AP MLO degradation.
- AP uplink congestion.
- AP PoE instability.
- AP reduced-power mode.
- High channel utilization.
- High retry rate.
- Low SNR.
- Mass client disconnect.
- Authentication failure.
- Certificate failure.
- DHCP failure.
- IPv6 addressing failure.
- DNS failure.
- Captive portal failure.
- Client connected but no application access.
- Roaming failure.
- MLO link failure.
- Controller HA failover.
- Controller capacity issue.
- Gateway-cluster failure.
- Vendor cloud-management disconnect.

Each workflow must:

1. Establish the affected scope.
2. Reconstruct event-time order.
3. Account for timestamp uncertainty.
4. Find the last known-good state.
5. Identify the first failed state.
6. Inspect AP, switch, VLAN, gateway, controller, AAA, PKI, DHCP, DNS, WAN, carrier, cloud, and application dependencies as applicable.
7. Gather only necessary evidence.
8. Check contradicting evidence.
9. Compare affected and unaffected peers.
10. Calculate confidence.
11. Recommend the least disruptive next action.
12. Define measurable success criteria.

25. Guarded auto-healing architecture

Do not allow Iris or another LLM to generate arbitrary CLI or API commands and execute them.

Create a registry of typed, pre-reviewed actions.

Each action must define:

- Action ID.
- Tenant ID.
- Target object type.
- Required provider capability.
- Required evidence.
- Contradiction checks.
- Blast-radius calculation.
- Risk level.
- Tenant policy.
- Approval policy.
- Idempotency key.
- Distributed target lock.
- Fencing token.
- Per-site concurrency budget.
- Per-controller concurrency budget.
- Maintenance-window check.
- Change-freeze check.
- Existing vendor-job check.
- Cooldown.
- Maximum attempts.
- Pre-action snapshot.
- Dry-run behavior.
- Preflight behavior.
- Execution behavior.
- Rollback action.
- Success criteria.
- Observation window.
- Vendor job ID.
- External audit reference.
- Complete audit fields.

Use the execution lifecycle:

Dry run

- Preflight
- Acquire distributed lock
- Obtain fencing token
- Recheck evidence
- Recheck blast radius
- Execute
- Observe
- Validate service recovery
- Roll back or escalate
- Release lock

The first production release must default to:

- Diagnose automatically.
- Recommend automatically.
- Collect approved diagnostics automatically.
- Execute customer-impacting remediation only after approval.

25.1 Observational actions

May run automatically, subject to authorization:

- Refresh AP state.
- Refresh controller state.
- Refresh gateway state.
- Re-run discovery.
- Query AP join state.
- Query AP control session.
- Query data-tunnel state.
- Query switch-port status.
- Query PoE status.
- Run bounded synthetic reachability tests.
- Run approved client-connectivity tests.
- Collect approved packet captures.
- Compare configuration.
- Gather diagnostic bundles.

25.2 Low-risk actions

May be automated under tenant policy:

- Re-establish a failed telemetry subscription.
- Refresh an expired integration token.
- Retry webhook registration.
- Replay failed discovery.
- Reconcile stale Correlix-side state.
- Clear only Correlix-owned stale cache.
- Re-run object and edge reconciliation.

25.3 Localized-impact actions

Require approval by default:

- Deauthenticate one client.
- Trigger one client reconnect.
- Restart one radio.
- Bounce one AP control session.
- Cycle PoE for one AP.

- Bounce one AP switch port.
- Move one AP to a secondary controller or gateway.

25.4 Broad-impact actions

Always require explicit approval:

- Restart an AP.
- Change RF channel.
- Change transmit power.
- Modify WLAN policy.
- Change VLAN assignment.
- Modify AAA settings.
- Change tunnel forwarding.
- Fail over a controller.
- Restart a controller process.
- Modify firewall or WAN policy.
- Change fabric or segmentation policy.

25.5 Prohibited autonomous actions

Never autonomously perform:

- Mass AP reboot.
- Controller reload.
- Gateway-cluster reload.
- Bulk channel changes.
- Bulk transmit-power changes.
- Security-policy bypass.
- Certificate-validation bypass.
- Disabling 802.1X.
- Disabling authentication controls.
- Unbounded command execution.
- Broad firewall, WAN, routing, fabric, or segmentation changes.

Do not recommend PoE cycling when several APs share a failing:

- Switch.
- Power source.
- Management VLAN.
- WAN path.
- Controller cluster.
- Gateway cluster.
- Site dependency.

Suppress localized remediation when a larger shared dependency is failing.

26. Remediation verification

A successful API or CLI response does not mean the incident is healed.

After every remediation, verify service-level recovery.

Depending on the incident, success may require:

- AP draws expected power.
- AP exits reduced-power mode.
- AP management interface becomes reachable.
- AP rejoins the control domain.
- Control session remains stable.
- Data tunnel becomes operational.
- Radios become operational.
- MLO links stabilize.
- Clients can associate.
- Authentication succeeds.
- Authorization succeeds.
- IPv4 or IPv6 addressing succeeds.
- DNS succeeds.
- Captive portal succeeds.
- Synthetic service probe passes.
- Original symptom remains absent for the observation window.
- No new affected scope appears.

If validation fails:

- Mark remediation unsuccessful.
- Roll back where possible.
- Prevent immediate repeated execution.
- Preserve all evidence.
- Escalate with a structured explanation.
- Do not declare recovery.

27. Multi-tenancy and security

All wireless resources must be tenant-scoped:

- Objects.
- Edges.
- Signals.
- Evidence.
- Client identities.
- Metrics.
- Logs.

- Packet captures.
- Provider credentials.
- Actions.
- Approvals.
- Audit records.

Review and test:

- ClickHouse row policies.
- PostgreSQL tenant predicates.
- API authorization.
- Cache-key tenant scoping.
- Redpanda message keys.
- Consumer filters.
- Background jobs.
- Remediation queues.
- Logs.
- Traces.
- Reports.
- Ticketing.
- Notifications.

Add negative tests proving one tenant cannot access another tenant's:

- AP inventory.
- Client sessions.
- User identities.
- RF data.
- Raw evidence.
- Provider credentials.
- Packet captures.
- Remediation actions.
- Action results.
- Controller details.
- Gateway details.
- WLAN policies.

Vendor credentials must use the existing secret-management abstraction.

Never place tokens, passwords, private keys, certificates, session cookies, or controller credentials in:

- Ordinary configuration tables.
- Logs.
- Traces.
- Error messages.
- Evidence payloads.
- Frontend responses.

28. Reliability requirements

All wireless collectors and providers must support:

- Pagination.
- Bounded concurrency.
- Request deadlines.
- Exponential backoff.
- Jitter.
- Vendor rate limits.
- `Retry-After`.
- Token expiration and refresh.
- Partial responses.
- Cursor continuation.
- Circuit breaking where appropriate.
- Backpressure.
- Duplicate delivery.
- Out-of-order delivery.
- Replay.
- Stale-event rejection.
- Idempotent writes.
- Dead-letter handling.
- Health metrics.
- Lag metrics.
- Schema evolution.
- Unknown enum preservation.

Discovery must not delete objects simply because one API poll omitted them.

Use lifecycle states and explicit missing-observation thresholds.

Differentiate:

- Missing from one poll.
- Temporarily unreachable.
- Administratively disabled.
- Retired.
- Replaced.
- Deleted.
- Unknown.
- Provider API incomplete.
- Provider permission insufficient.

29. Testing strategy

Test the failure class, not only individual examples.

29.1 Unit tests

Add tests for:

- Canonical normalization.
- Stable object identity.
- Tenant-scoped identity.
- Field-level provenance.
- Conflict resolution.
- Forwarding-mode mapping.
- Encapsulation-stack mapping.
- Controller-cluster mapping.
- MLO session mapping.
- Temporal edges.
- Roaming history.
- Onboarding episode transitions.
- Applicable and skipped stages.
- Signature predicates.
- Contradiction checks.
- Timestamp uncertainty.
- Confidence calculation.
- Remediation preconditions.
- Idempotency.
- Distributed locking.
- Tenant authorization.

29.2 Provider contract tests

Every wireless provider must pass the same applicable contract suite covering:

- Stable IDs.
- Pagination.
- Rate limiting.
- Authentication failure.
- Token refresh.
- Missing fields.
- Unknown enum values.
- Duplicate events.
- Out-of-order events.
- Delayed events.
- Schema evolution.
- Clock skew.
- Timestamp uncertainty.
- Controller failover.
- Gateway failover.
- AP replacement.
- Client roaming.

- MLO link changes.
- Partial API failure.
- Unsupported capabilities.

29.3 Golden fixtures

Store sanitized payloads under paths consistent with repository conventions, conceptually similar to:

- `testdata/wireless/cisco9800/`
- `testdata/wireless/aruba/`
- `testdata/wireless/mist/`
- `testdata/wireless/meraki/`

Each fixture must include:

- Raw vendor payload.
- Expected canonical observation.
- Expected objects.
- Expected edges.
- Expected metrics.
- Expected evidence.
- Expected signature.
- Expected verdict.
- Expected provenance.
- Expected timestamp behavior.

29.4 Real integration tests

Exercise actual data paths rather than source scans only.

Verify:

- Event-bus serialization and replay.
- ClickHouse writes and reads.
- PostgreSQL constraints.
- Redis keys.
- Tenant row policies.
- API responses.
- Frontend topology responses.
- Correlation execution.
- Evidence retrieval.
- Correlation-object lifecycle.
- Resolution and reopen behavior.
- Notification and ticketing behavior.
- Action queue.
- Action audit history.
- Remediation validation.

29.5 Fault injection

Create lab scenarios for:

1. AP PoE removal.
2. Insufficient PoE.
3. AP switch-port shutdown.
4. LAG member failure.
5. Access-switch failure.
6. AP management-VLAN failure.
7. AP default-gateway failure.
8. CAPWAP or control-traffic block.
9. Central data-tunnel block.
10. Controller-member failure.
11. Controller-cluster failure.
12. Gateway-member failure.
13. Gateway-cluster failure.
14. Controller HA failover.
15. RADIUS latency.
16. RADIUS timeout.
17. RADIUS rejection.
18. Certificate failure.
19. DHCPv4 failure.
20. DHCPv6 failure.
21. SLAAC failure.
22. DNS failure.
23. Captive-portal failure.
24. RF interference.
25. Channel congestion.
26. AP reboot.
27. AP software failure.
28. MLO link failure.
29. Telemetry failure without customer impact.
30. Vendor API outage without forwarding impact.
31. Cloud-management outage without local forwarding impact.
32. SD-WAN impairment.
33. ISP circuit impairment.
34. AWS Direct Connect impairment.
35. Azure ExpressRoute impairment.
36. Application degradation affecting wired and wireless clients.

The system must not falsely attribute scenarios 29 through 36 to wireless when evidence points elsewhere.

30. Required invariants

Enforce these through implementation and tests:

- No cross-tenant data leakage.
- No remediation without an idempotency key.
- No remediation without a distributed target lock.
- No confirmed RCA based on receive-time ordering alone.
- No confirmed RCA based on temporal proximity alone.
- No customer-impacting action without blast-radius calculation.
- No action marked recovered before post-action validation.
- No unreviewed arbitrary command execution.
- No raw vendor enum silently enters canonical schemas.
- No object deletion from one missing poll.
- No stale event reopens a resolved incident without state-machine justification.
- No vendor provider directly mutates canonical correlation state outside approved interfaces.
- No Iris narrative claims higher confidence than the structured verdict.
- No SSID string is used as the sole WLAN identity.
- No assumption that one client session has only one wireless link.
- No assumption that one controller member represents the entire cluster.
- No assumption that cloud-management loss equals forwarding loss.
- No assumption that control-session health proves data-path health.
- No source timestamp is overwritten.

31. Observability

Instrument the wireless subsystem with:

- Discovery duration.
- Discovery failures.
- Vendor API requests.
- Vendor API errors.
- Rate-limit events.
- Token refreshes.
- Cursor lag.
- Event-processing lag.
- Event-time versus receive-time skew.
- Timestamp uncertainty.
- Normalization failures.
- Unknown vendor enums.
- Unresolved identities.
- Provenance conflicts.
- Objects by type.
- Stale objects.
- Duplicate observations.
- Dropped observations.

- Edge-reconciliation failures.
- Session count.
- Active MLO link count.
- Onboarding-stage duration.
- Signature evaluations.
- Verdict distribution.
- Undetermined-verdict rate.
- Remediation attempts.
- Approval latency.
- Remediation success.
- Remediation failure.
- Rollback count.
- Post-action validation failure.

Use existing Correlix conventions for:

- Metric naming.
- Logging.
- Tracing.
- Dashboards.
- Alerts.
- Tenant labels.
- Cardinality controls.

32. API and UI integration

Integrate wireless into existing Correlix APIs and topology views.

Users should be able to navigate:

Client

- Session
- Client link or MLO links
- BSSID
- Radio
- AP
- AP interface
- Access-switch port
- Access switch
- Controller, gateway, or data tunnel when applicable
- LAN
- SD-WAN or WAN
- Carrier
- Cloud or DC
- Application

Provide separate graph overlays for:

- Client service path.
- AP control dependencies.
- AP management path.
- RF relationships.
- Authentication dependencies.
- DHCP and DNS dependencies.
- MLO links.
- Tunnel and encapsulation path.

Do not render all relationship types simultaneously if that produces an unreadable graph.

Expose:

- Client onboarding timeline.
- Applicable onboarding stages.
- Last successful stage.
- First failed stage.
- AP state.
- AP power state.
- Control-session state.
- Data-tunnel state.
- Forwarding mode.
- Encapsulation stack.
- Controller or gateway cluster state.
- Controller or gateway member state.
- MLO link state.
- Supporting evidence.
- Contradicting evidence.
- Missing evidence.
- Root-cause confidence.
- Responsibility seam.
- Affected scope.
- Troubleshooting workflow.
- Remediation eligibility.
- Approval status.
- Action history.
- Recovery-validation status.

Do not load every historical client into the default topology view.

Use on-demand or incident-scoped materialization.

33. Initial production scope

Do not attempt every wireless feature in one implementation stream.

The first production milestone should focus on:

- Cisco Catalyst 9800 read-only integration.
- Controller and cluster inventory.
- AP inventory.
- Radio inventory.
- SSID, WLAN, and BSSID inventory.
- AP-to-switch-port topology.
- AP power and uplink relationships.
- AP control-session health.
- Local versus central forwarding.
- Client sessions.
- Basic MLO-compatible schemas.
- Applicability-aware onboarding episodes.
- AP offline RCA.
- Authentication RCA.
- DHCP RCA.
- Client connected but no service RCA.
- Cross-seam root-cause exclusion.
- Iris structured troubleshooting.
- No autonomous customer-impacting remediation.

Defer until the core model is proven:

- Rogue and WIPS analytics.
- Location analytics.
- BLE and IoT analytics.
- Full spectrum analysis.
- Automatic RF channel changes.
- Automatic transmit-power changes.
- Bulk remediation.
- Large-scale packet-capture orchestration.
- Full historical client topology visualization.
- Mass AP actions.

34. Recommended delivery phases

Deliver small, reviewable phases.

Phase 0: Repository analysis and architecture

- Repository-impact map.
- Architecture decision records.
- Existing correlation-engine integration map.
- Canonical object model.
- Canonical edge model.

- Observation schema.
- MLO model.
- WLAN, SSID, and BSSID model.
- Cluster model.
- Capability model.
- Storage model.
- Migration plan.
- Test plan.
- Live-validation plan.

Phase 1: Canonical schemas

- Object types.
- Edge types.
- Event families.
- Database migrations.
- Serialization.
- API compatibility.
- Tenant protections.
- Timestamp provenance.
- Field provenance.

Phase 2: Cisco Catalyst 9800 read-only discovery

- Controller inventory.
- Controller-cluster inventory.
- AP inventory.
- AP interfaces.
- Radios.
- SSIDs.
- WLANs.
- BSSIDs.
- AP join state.
- Control sessions.
- Client sessions.
- Forwarding mode.
- MLO-compatible representation.

Phase 3: Topology and path integration

- AP-to-switch-port reconciliation.
- PoE relationship.
- Management path.
- Controller path.
- Gateway path.
- Local forwarding.
- Central forwarding.

- Existing LAN-seam attachment.
- Ordered path-hop integration.

Phase 4: Client onboarding

- Applicability-aware episode.
- Authentication.
- Authorization.
- IPv4 and IPv6 addressing.
- DHCP.
- DNS.
- Failure-stage detection.
- Client timeline.

Phase 5: Wireless correlation

- AP offline.
- Mass AP failure.
- Control-versus-data-path failure.
- Onboarding failure.
- Performance degradation.
- Roaming failure.
- MLO degradation.
- Cross-seam root-cause exclusion.

Phase 6: Iris troubleshooting

- Structured diagnosis.
- Evidence explanation.
- Contradiction checking.
- Counterfactual checking.
- Diagnostic workflows.
- Undetermined-verdict handling.

Phase 7: Multi-vendor proof

- Aruba provider or skeleton.
- Gateway-cluster model.
- Cloud-managed architecture validation.
- Provider contract tests.

Phase 8: Guarded diagnostics and remediation framework

- Typed action registry.
- Distributed locks.
- Fencing.
- Approval policies.

- Idempotency.
- Audit trail.
- Post-action validation.
- Rollback.
- Recommendation-only default.

Phase 9: Fault injection and production hardening

- Physical or simulated failure testing.
- Scale tests.
- Replay tests.
- Security tests.
- Tenant-isolation tests.
- Rollout dashboards.
- Operational runbooks.

35. Required deliverables for every phase

For every phase, report:

- Design decisions.
- Exact files changed.
- Packages changed.
- Database changes.
- Event-schema changes.
- API changes.
- UI changes.
- Correlation-engine changes.
- Backward-compatibility impact.
- Tests added.
- Tests executed.
- Results.
- Remaining untested behavior.
- Failure modes.
- Feature flag.
- Rollout plan.
- Rollback plan.
- Operational metrics.
- Known limitations.
- Approval decisions required.

Clearly label each conclusion as one of:

- Verified in production code.
- Verified by an existing test.
- Verified by a new test.
- Verified by a recorded fixture.

- Supported by vendor documentation.
- Architectural inference.
- Unverified assumption.
- Requires simulator validation.
- Requires live-controller validation.
- Requires physical fault injection.

36. Mandatory implementation stop gate

After repository investigation, stop and return the architecture and impact report.

Do not implement broad production changes until the report demonstrates:

1. Exact reuse of existing Correlix discovery abstractions.
2. Exact reuse of the current Correlix correlation lifecycle.
3. Exact mapping into correlation signals, objects, edges, evidence, and verdicts.
4. Exact incident merge, resolution, and reopen behavior.
5. Exact database and migration impact.
6. Correct event-time and timestamp-uncertainty handling.
7. Wi-Fi 7 MLO compatibility.
8. Separate SSID, WLAN, and BSSID identities.
9. Local, central, and mixed forwarding compatibility.
10. Controller-cluster and gateway-cluster compatibility.
11. Cloud-managed compatibility.
12. Ordered service-path integration.
13. Storage and cardinality feasibility.
14. Tenant-isolation enforcement.
15. Remediation safety architecture.
16. A realistic live-controller validation plan.
17. A realistic physical fault-injection plan.

Do not proceed merely because the proposed architecture appears reasonable.

37. Initial task to perform now

Begin with repository investigation and architecture only.

Do not start broad production implementation during this first pass.

Return:

1. A concise summary of the current Correlix architecture relevant to wireless.
2. The exact current Correlix correlation-engine lifecycle.
3. Existing abstractions that can be reused.
4. Existing abstractions that require extension.

5. Architectural gaps and blockers.
6. Proposed relationship between Wireless Access and the existing LAN seam.
7. Exact canonical object schema.
8. Exact canonical edge schema.
9. Exact SSID, WLAN, and BSSID model.
10. Exact Wi-Fi 7 MLO model.
11. Exact controller, gateway, cluster, and member model.
12. Exact observation and temporal-provenance schema.
13. Proposed provider interfaces and capability model.
14. Cisco Catalyst 9800 source and integration plan.
15. Multi-vendor architecture-validation plan.
16. Client onboarding-episode model.
17. Wireless correlation-signature model.
18. Iris troubleshooting architecture.
19. Guarded remediation architecture.
20. High-cardinality storage and retention plan.
21. Database and migration impact.
22. Exact repository files and packages expected to change.
23. Test and fault-injection plan.
24. Phased delivery plan.
25. Open questions requiring approval.

Be brutally honest.

Do not claim a feature is production-ready merely because it compiles or passes mocked tests.

The final architecture must demonstrate that Correlix can represent all of the following without corrupting the canonical model:

- Cisco controller-based CAPWAP.
- Aruba gateway-tunneled wireless.
- Cloud-managed locally forwarding wireless.
- Controllerless wireless.
- Local switching.
- Central switching.
- Mixed forwarding.
- Split tunneling.
- Nested encapsulation.
- AP management paths separate from control paths.
- AP control paths separate from client data paths.
- Controller clusters separate from physical members.
- Gateway clusters separate from physical members.
- Wi-Fi 7 Multi-Link Operation.
- IPv4, IPv6, and dual-stack onboarding.
- End-to-end correlation from wireless client through LAN, SD-WAN, carrier, cloud, DC, and application.
- Wireless as root cause, contributor, affected domain, unrelated domain, or undetermined.

Prefer an UNDETERMINED diagnosis over unsupported certainty.